

### Retropropagación (Backpropagation)

#### Observación práctica

- Una red más profunda y estrecha suele generalizar mejor que una más ancha y shallow
- El mismo número de parámetros distribuido en más capas → mejor performance

#### Objetivo

- Actualizar los parámetros de una red para reducir la función de pérdidas  $J$ :

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial J}{\partial \theta}$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Backpropagation

#### ¿Cómo calcular $\partial J / \partial W$ para cada capa?

- La pérdida  $J$  depende de la salida, que depende de la última capa
- La última capa depende de la penúltima, y así sucesivamente
- Aplicación de la regla de la cadena al grafo computacional de una red neuronal para propagar la señal de error desde la salida hacia los pesos de las primeras capas
- Complejidad computacional:  $O(\text{parámetros})$  — igual que un forward pass

#### Dos operaciones principales

- Forward pass: calcular y guardar las activaciones de cada capa
- Backward pass: propagar gradientes desde la salida hacia la entrada

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Backpropagación Vectorizada

- En la práctica trabajamos con matrices (batches de  $N$  muestras)

#### Notación

- $H^l \in \mathbb{R}^{N \times H_l}$  — activaciones de la capa  $l$  para el batch
- $W^l \in \mathbb{R}^{H_l \times H_{l-1}}$  — pesos de la capa  $l$
- $\delta^l \equiv \frac{\partial J}{\partial Z^l} \in \mathbb{R}^{N \times H_l}$  — gradiente de la pérdida respecto a las preactivaciones

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Algoritmo Backpropagation Vectorizado

#### FORWARD PASS — guardar activaciones

Entrada (capa 0):

$$H^0 = X$$

Preactivación capa  $l$  :

$$Z^l = H^{l-1} W^{l\top} + b^l$$

Activación capa  $l$  ( $f$  es la función de activación):

$$H^l = f(Z^l)$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Algoritmo Backpropagation Vectorizado

#### BACKWARD PASS — propagar gradientes

Propagación del error hacia la capa anterior ( $\odot$  = Hadamard):

$$\delta^l = (\delta^{l+1} W^{l+1}) \odot f'(Z^l)$$

Gradiente de la pérdida respecto a los pesos  $W^l$ :

$$\frac{\partial L}{\partial W^l} = \delta^{l\top} H^{l-1}$$

Gradiente de la pérdida respecto al bias  $b^l$  (suma sobre el batch):

$$\frac{\partial L}{\partial b^l} = \sum_n \delta_n^l$$

Gradiente respecto a las activaciones de la capa anterior:

$$\frac{\partial L}{\partial H^{l-1}} = \delta^l W^l$$

Aplicar derivada de la activación para obtener  $\delta^l$ :

$$\delta^l = \left( \frac{\partial L}{\partial H^l} \right) \odot f'(Z^l)$$

### **Algoritmo Backpropagation Vectorizado** **ACTUALIZACIÓN (Gradient Descent)**

Actualización de pesos:

$$W^l \leftarrow W^l - \eta \frac{\partial L}{\partial W^l}$$

Actualización de bias:

$$b^l \leftarrow b^l - \eta \frac{\partial L}{\partial b^l}$$

### Implementación de un MLP en PyTorch

#### **nn.Module — La Base de PyTorch DL**

- nn.Module es la clase base para todos los modelos en PyTorch

#### **Responsabilidades de nn.Module**

- Contiene los parámetros del modelo (nn.Parameter)
- Define el grafo computacional en forward()
- Gestiona sub-módulos jerárquicamente (.children(), .modules())
- Permite mover el modelo entre dispositivos (.to(device), .cuda())
- Alternancia entre modos entrenamiento y evaluación (.train(), .eval())
- Serialización: guardar y cargar pesos (.state\_dict(), .load\_state\_dict())

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Implementación de un MLP en PyTorch

### nn.Module — La Base de PyTorch DL

### Capas predefinidas más comunes

- `nn.Linear(in, out)`: capa completamente conectada
- `nn.ReLU()`, `nn.GELU()`, `nn.SiLU()`: funciones de activación
- `nn.BatchNorm1d(features)`: batch normalization
- `nn.Dropout(p)`: dropout
- `nn.Sequential`: apila capas en orden secuencial



# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### **Regularización — Motivación**

- Las redes neuronales profundas tienen millones de parámetros: alto riesgo de overfitting

### **Trade-off Bias-Varianza en DL**

- Underfitting (alto bias): modelo demasiado simple, error alto en train y test
- Overfitting (alta varianza): memoriza el training set, falla en test

### **Señales de overfitting**

- Brecha grande entre train loss y validation loss
- Validation loss sube mientras train loss sigue bajando
- Alta accuracy en train, baja en validation

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Regularización — Motivación

#### Estrategias de regularización

- Restricción de parámetros: L1/L2 weight decay
- Ruido durante entrenamiento: Dropout
- Normalización de activaciones: Batch Norm, Layer Norm
- Data augmentation
- Early stopping: parar cuando val\_loss deja de mejorar

### Regularización L2 — Weight Decay

Función de pérdida regularizada con L2 (norma de Frobenius)

$$L_{reg} = L + \frac{\lambda}{2} \sum_l \|W^l\|^2$$

donde  $\lambda$  es el coeficiente de regularización (hiperparámetro)

**Gradiente**

$$\frac{\partial L_{reg}}{\partial W^l} = \frac{\partial L}{\partial W^l} + \lambda W^l$$

**Actualización** — factor  $(1 - \eta\lambda)$  produce el efecto "weight decay":

$$W^l \leftarrow (1 - \eta\lambda)W^l - \eta \frac{\partial L}{\partial W^l}$$

**Interpretación**

- Penaliza pesos grandes: los fuerza a mantenerse pequeños
- Promueve soluciones suaves y generalizables

### Regularización L1 — Sparsity

#### Función de pérdida regularizada con L1

$$L_{reg} = L + \lambda \sum_{i,j} |w_{ij}|$$

Gradiente L1 — penalización constante (no depende del tamaño del peso):

$$\frac{\partial L_{reg}}{\partial w_{ij}} = \frac{\partial L}{\partial w_{ij}} + \lambda \cdot \text{sign}(w_{ij})$$

#### Propiedad de sparsity

- L1 empuja los pesos exactamente a cero → soluciones sparse
- L2 reduce los pesos pero rara vez los lleva a exactamente cero

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Comparativa L1 vs L2

- L1: sparse, selección implícita de features, menos estable con SGD
- L2: pesos pequeños pero no nulos, más estable, equivalente a weight decay

### Uso en DL

- L2 (weight decay) es estándar en casi todos los modelos modernos
- L1 se usa menos en DL profundo — más común en regresión lineal/logística (Lasso)

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Dropout (Srivastava et al., 2014)

#### Mecanismo

- Durante entrenamiento: cada neurona se desactiva independientemente con probabilidad  $p$
- Las neuronas activas se escalan por  $1/(1-p)$  para mantener la esperanza
- Durante evaluación: todas las neuronas activas — no hay dropout

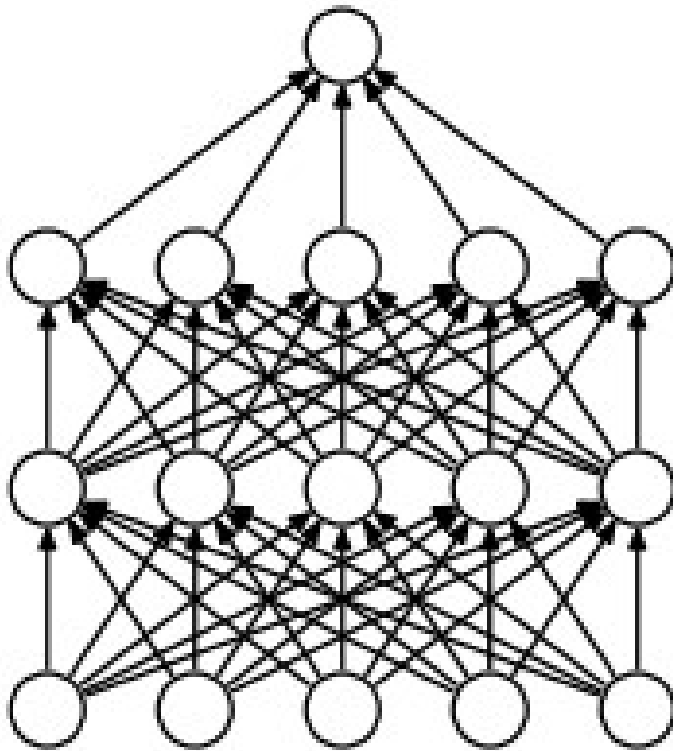
#### Implementación

- `model.train()`: activa el dropout
- `model.eval()`: desactiva el dropout — (No olvidarlo)

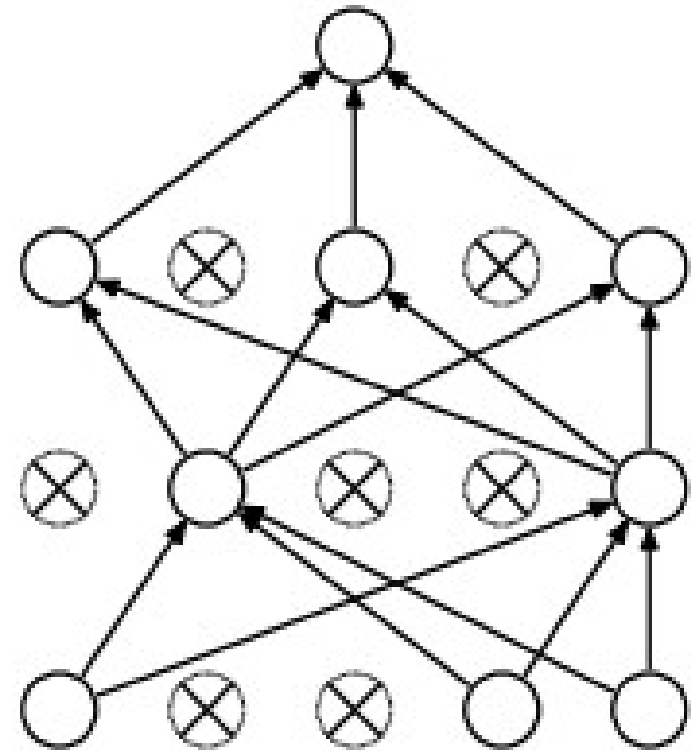
# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Dropout (Srivastava et al., 2014)



Red neuronal estándar



Red con dropout aplicado

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Dropout

#### ¿Por qué funciona? — Interpretaciones

- Ensemble: entrena simultáneamente  $2^n$  sub-redes — evaluación es el promedio implícito
- Ruido: añade ruido aleatorio que previene la co-adaptación de neuronas
- Robustez: la red aprende a no depender de ninguna neurona específica

#### Tasas típicas de dropout

- Capas densas:  $p = 0.3 - 0.5$
- Capas de entrada:  $p = 0.1 - 0.2$



# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### **Batch Normalization (Ioffe & Szegedy, 2015)**

#### **Problema que resuelve: Internal Covariate Shift**

- La distribución de activaciones de cada capa cambia durante el entrenamiento
- Las capas posteriores deben adaptarse constantemente a distribuciones variables
- Esto frena el entrenamiento y requiere learning rates pequeños

#### **Beneficios**

- Permite learning rates más grandes → entrenamiento más rápido
- Regularización implícita → menos necesidad de dropout
- Reduce sensibilidad a la inicialización

#### **¿Antes o después de la activación?**

- Original (Ioffe & Szegedy): BN antes de la activación
- Pre-Norm (práctica moderna): BN antes de cada bloque, activación después

### Batch Normalization (Ioffe & Szegedy, 2015)

#### Algoritmo — por batch y por feature

Media del mini-batch (N muestras):

$$\mu_B = \frac{1}{N} \sum_n z_n$$

Varianza del mini-batch:

$$\sigma_B^2 = \frac{1}{N} \sum_n (z_n - \mu_B)^2$$

Normalización ( $\epsilon$  para estabilidad numérica):

$$\widehat{z}_n = \frac{z_n - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Escalado y desplazamiento (parámetros aprendibles  $\gamma$ ,  $\beta$ ):

$$y_n = \gamma \cdot \widehat{z}_n + \beta$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Batch Normalization — Detalles Importantes

#### Durante entrenamiento

- $\mu$  y  $\sigma^2$  se calculan sobre el mini-batch actual
- Se actualiza un running mean y running var (media exponencial móvil)

#### Durante evaluación (`model.eval()`)

- Se usan los running mean y running var acumulados durante el entrenamiento
- Si se olvida `model.eval()`: BN usa stats del batch de test → resultados incorrectos

#### Parámetros aprendibles ( $\gamma$ , $\beta$ )

- $\gamma$  (scale) y  $\beta$  (shift): permiten que la red revierta la normalización si es necesario
- Inicialmente:  $\gamma = 1$ ,  $\beta = 0$

### Batch Normalization — Detalles Importantes

#### Limitación principal

- Depende del tamaño del batch: batch pequeño → estimaciones de  $\mu, \sigma^2$  ruidosas
- Con `batch_size=1`: totalmente inestable
- Alternativa: Layer Normalization (independiente del batch)

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Layer Normalization (Ba et al., 2016)

#### Diferencia clave con Batch Norm

- BN normaliza a través del batch ( $\text{dim}=0$ ): calcula  $\mu, \sigma$  sobre las  $N$  muestras
- LN normaliza a través de los features ( $\text{dim}=-1$ ): calcula  $\mu, \sigma$  para cada muestra

#### Ventajas sobre Batch Norm

- Independiente del tamaño del batch — funciona con `batch_size=1`
- Idéntico en training y evaluation — sin modo dual
- Estándar en todos los Transformers (BERT, GPT, ViT)

#### ¿Cuándo usar cuál?

- Batch Norm: CNNs, MLPs con batch grande — práctica común en visión
- Layer Norm: Transformers, RNNs, NLP — cualquier arquitectura con batch variable

### Layer Normalization (Ba et al., 2016)

#### Algoritmo

Media por muestra (sobre los H features):

$$\mu_n = \frac{1}{H} \sum_h z_{nh}$$

Varianza por muestra:

$$\sigma_n^2 = \frac{1}{H} \sum_h (z_{nh} - \mu_n)^2$$

Normalización:

$$\widehat{z}_{nh} = \frac{z_{nh} - \mu_n}{\sqrt{\sigma_n^2 + \varepsilon}}$$

Escalado y desplazamiento ( $\gamma_h, \beta_h$  son parámetros por feature):

$$y_{nh} = \gamma_h \cdot \widehat{z}_{nh} + \beta_h$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Gradient Descent — Revisión y Limitaciones

#### SGD básico

- $\theta_{t+1} = \theta_t - \eta \cdot g_t$  donde  $g_t = \partial L / \partial \theta$
- Problema: muy sensible al learning rate
- Problema: misma LR para todos los parámetros — ineficiente
- Problema: oscila en valles alargados (plateaus y ravines)

#### Problemas

- Saddle points: más comunes que mínimos locales en alta dimensión
- Ravines: curvatura alta en una dirección, baja en otra → zigzag lento
- Plateaus: gradientes casi nulos durante largas regiones

#### Soluciones

- Momentum: acumular historia de gradientes para suavizar trayectoria
- Adaptive learning rates: ajustar  $\eta$  por parámetro según su historial

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### SGD con Momentum

#### Algoritmo

Velocidad — media exponencial móvil del gradiente:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t$$

Actualización con momentum:

$$\theta_{t+1} = \theta_t - \eta \cdot v_t$$

donde  $\beta$  es el coeficiente momentum

#### Intuición física

- La pelota acumula velocidad mientras rueda cuesta abajo
- En direcciones consistentes: velocidad aumenta, convergencia más rápida
- En direcciones oscilantes: la velocidad se cancela, reducción del zigzag

#### Valores típicos

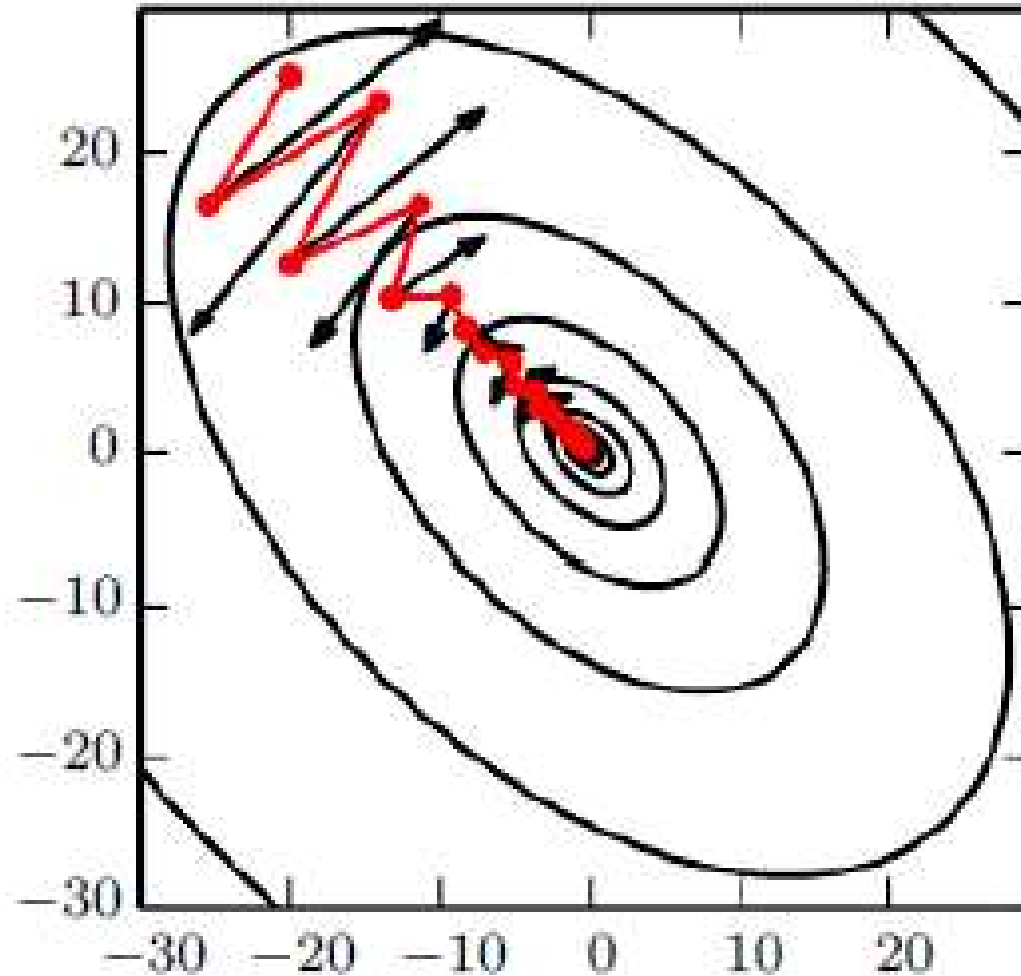
- $\eta = 0.01 - 0.1$  (mucho mayor que sin momentum)
- $\beta = 0.9$  — valor estándar en casi todos los trabajos



# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### SGD con Momentum



### **RMSprop — Adaptive Learning Rates**

#### **RMSprop (Hinton, 2012)**

Media exponencial del cuadrado del gradiente ( $\rho \approx 0.9$ ):

$$s_t = \rho \cdot s_{t-1} + (1 - \rho) \cdot g_t^2$$

Actualización con learning rate adaptativa:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \varepsilon} \cdot g_t$$

Efecto: LR grande para parámetros con gradientes pequeños y viceversa

#### **Cuándo usar RMSprop**

- Útil para RNNs (gradientes muy variables en el tiempo)
- En la práctica: Adam (que combina momentum + RMSprop) es más popular

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### **Adam — Adaptive Moment Estimation**

**Kingma & Ba (2014) — el optimizador más usado en DL**

- Combina momentum (primer momento) con RMSprop (segundo momento)

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Adam — Adaptive Moment Estimation

#### Algoritmo

Primer momento — media exponencial de gradientes ( $\beta_1 = 0.9$ ):

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

Segundo momento — media del cuadrado del gradiente ( $\beta_2 = 0.999$ ):

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

Corrección de bias — primer momento (importante al inicio del entrenamiento):

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

Corrección de bias — segundo momento:

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Actualización Adam ( $\eta = 10^{-3}$  por defecto,  $\varepsilon = 10^{-8}$ ):

$$\theta_{t+1} = \theta_t - \frac{\eta \cdot \widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Adam — Adaptive Moment Estimation

#### Hiperparámetros por defecto (rara vez se cambian)

- $\beta_1 = 0.9$  (momentum),  $\beta_2 = 0.999$  (escala),  $\varepsilon = 10^{-8}$
- $\eta = 10^{-3}$  — valor estándar de partida

#### Intuición

- $\widehat{m}_t$  : dirección suavizada del descenso
- $\sqrt{\widehat{v}_t}$ : escala adaptativa — parámetros con gradientes grandes tienen LR efectiva menor

### AdamW — Adam con Weight Decay Correcto

#### El problema con Adam + L2

- En Adam, añadir  $\lambda ||W||^2$  a la pérdida e incluirlo en  $g_t$  no equivale a weight decay puro
- El L2 en Adam queda amortiguado por  $\sqrt{\widehat{v}_t} \rightarrow$  diferentes parámetros reciben decay diferente

#### AdamW (Loshchilov & Hutter, 2019) — la corrección

- Separar el weight decay de la actualización de Adam:
- Actualización AdamW — el weight decay actúa directamente sobre  $\theta$ :

$$\theta_{t+1} = \theta_t - \frac{\eta \cdot \widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon} - \eta \cdot \lambda \cdot \theta_t$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### **AdamW — Adam con Weight Decay Correcto**

#### **Impacto práctico**

- AdamW generaliza significativamente mejor que Adam con L2 en la pérdida
- Adoptado como estándar en: GPT, BERT, ViT, ResNet modernos, YOLO

#### **Guía de uso**

- Empezar siempre con AdamW — es raro que SGD o Adam puro supere a AdamW
- `weight_decay`: 0.01-0.1 para modelos grandes;  $1e-4$  para modelos pequeños

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Comparativa de Optimizadores

| Optimizador  | LR adaptativa | Momentum | Weight Decay | Uso típico                        |
|--------------|---------------|----------|--------------|-----------------------------------|
| SGD          | No            | Opcional | Vía L2       | CNNs clásicas, ImageNet           |
| SGD+Nesterov | No            | Sí       | Vía L2       | CV con batch grande               |
| RMSprop      | Sí            | No       | Vía L2       | RNNs, RL                          |
| Adam         | Sí            | Sí       | Problemático | NLP, atención, prototipos         |
| AdamW        | Sí            | Sí       | ✓ Correcto   | Estándar moderno (GPT, ViT, YOLO) |

### Recomendación práctica

- AdamW como punto de partida en prácticamente todos los proyectos
- SGD+momentum para CNNs en visión con LR scheduling agresivo (OneCycle, Cosine)



# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Learning Rate Scheduling

#### ¿Por qué no usar un LR fijo?

- LR grande al inicio: converge rápido pero puede oscilar cerca del mínimo
- LR pequeño al inicio: muy lento, puede quedarse atrapado en plateaus
- Solución: LR grande al inicio, reducir progresivamente al acercarse al mínimo

### Learning Rate Scheduling

#### Estrategias principales

- Step decay: reducir LR por factor  $\gamma$  cada  $N$  épocas
- Cosine Annealing: LR sigue un coseno de  $\eta_{max}$  a  $\eta_{min}$
- OneCycleLR: aumentar hasta  $\eta_{max}$  y luego bajar (Smith, 2019)
- Warmup lineal: LR crece linealmente de 0 a  $\eta_{max}$  en los primeros  $K$  steps
- ReduceLROnPlateau: reduce LR cuando la val\_loss no mejora

#### Warmup + Cosine Decay (estándar en Transformers y ViT)

- Fase 1 (warmup): LR crece linealmente de 0 a  $\eta_{max}$  en  $T_{warmup}$  steps
- Fase 2: LR sigue coseno decreciente hasta  $\eta_{min}$
- Evita inestabilidad al inicio cuando los pesos aún son aleatorios

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### ¿Por qué Importa la Inicialización?

#### Dos extremos problemáticos

- Pesos demasiado grandes: activaciones explotan → saturación o gradientes explosivos
- Pesos demasiado pequeños: activaciones se desvanecen → vanishing gradients

#### Mala inicialización → fracaso del entrenamiento

- Redes de 10+ capas con pesos aleatorios  $N(0,1)$ : training imposible sin BN
- Con buena inicialización: el entrenamiento puede comenzar de forma estable

#### El objetivo

- La varianza de las activaciones debe mantenerse aproximadamente constante capa a capa
- La varianza de los gradientes también debe mantenerse al propagar hacia atrás

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Inicialización He / Kaiming

#### He et al. (2015) — 'Delving deep into rectifiers'

- Diseñada específicamente para ReLU y variantes

#### Algoritmo

Varianza He — compensa el factor  $\frac{1}{2}$  de ReLU ( $\approx 50\%$  de neuronas desactivadas):

$$Var(w) = \frac{2}{D_{in}}$$

Distribución uniforme He:

$$w \sim Uniform\left(-\sqrt{\frac{6}{D_{in}}}, +\sqrt{\frac{6}{D_{in}}}\right)$$

Distribución normal He:

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{D_{in}}}\right)$$

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Gradient Flow — Diagnóstico

- Monitorear el flujo de gradientes es crítico para detectar problemas de entrenamiento

#### ¿Qué monitorear?

- Norma del gradiente por capa:  $||\partial L / \partial W^1||$
- Histograma de gradientes: distribución por capa y por época
- Ratio gradiente/peso:  $||\partial L / \partial W^1|| / ||W^1||$  — debe ser  $\sim 10^{-3}$

### Gradient Flow — Diagnóstico

#### Señales de vanishing gradient

- Las primeras capas tienen gradientes órdenes de magnitud menores que las últimas
- La pérdida no disminuye aunque el modelo tenga capacidad suficiente
- Los pesos de las primeras capas no cambian entre épocas

#### Señales de exploding gradient

- La pérdida diverge (NaN o Inf)
- Los pesos crecen sin control entre épocas
- Solución: gradient clipping

$$g_t \leftarrow g_t \cdot \min\left(1, \frac{\text{max\_norm}}{\|g_t\|}\right)$$

#### Herramienta

- W&B: gradient histograms en tiempo real durante el entrenamiento

### Sanity Checks — Antes de Entrenar

- Antes de lanzar un entrenamiento largo, verificar que todo funciona correctamente

#### Check 1: pérdida inicial correcta

- Con softmax: pérdida inicial esperada  $\approx \log(K)$  para  $K$  clases
- Con  $K=10$  (MNIST):  $\log(10) \approx 2.30$  — si la pérdida inicial es muy distinta, hay un bug

#### Check 2: overfit en un batch pequeño

- Entrenar solo en 5-20 muestras durante muchas épocas
- El modelo debe lograr training loss  $\approx 0$  — si no, hay un bug en el modelo o en la pérdida

### Sanity Checks — Antes de Entrenar

#### Check 3: verificar shapes

- Imprimir shapes de X, y, logits, loss en el primer batch
- Verificar que el forward pass produce la shape esperada

#### Check 4: gradient check numérico

- `torch.autograd.gradcheck()` — compara gradientes analíticos con diferencias finitas
- Útil cuando se implementan capas propias

#### Check 5: monitorear la primera época

- La pérdida debe bajar monotónicamente en el primer batch si el LR es correcto



# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Diagnóstico — Problemas Comunes

#### La pérdida no baja

- LR demasiado pequeño → aumentar 10x y ver si hay movimiento
- Gradientes cerca de 0 → revisar funciones de activación e inicialización
- Bug en la pérdida → sanity check con batch pequeño

#### La pérdida diverge o sale NaN

- LR demasiado grande → reducir 10x
- Gradientes exploding → aplicar gradient clipping (max\_norm=1.0)
- Overflow en  $\exp()$  → usar log-sum-exp trick, revisar softmax con LogSoftmax

# INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

## Unidad II

### Diagnóstico — Problemas Comunes

#### Overfitting

- Aumentar dropout, añadir weight decay, reducir capacidad del modelo
- Data augmentation, más datos

#### Underfitting

- Aumentar capacidad: más capas, más neuronas
- Reducir regularización, entrenar más épocas
- Revisar si hay bug en el preprocesamiento de datos

#### Entrenamiento lento

- Verificar que los datos están en GPU, no solo el modelo
- Aumentar batch\_size si la VRAM lo permite